Contact Management System

Comprehensive Project Documentation • Generated on February 20, 2026

Project Summary: A full-stack contact management application with JWT-based authentication. Users can register, log in, and manage contacts with user-level data isolation.

Frontend: React + Vite | Backend: Spring Boot 3 | Database: MySQL | Security: JWT + Spring Security

Table of Contents

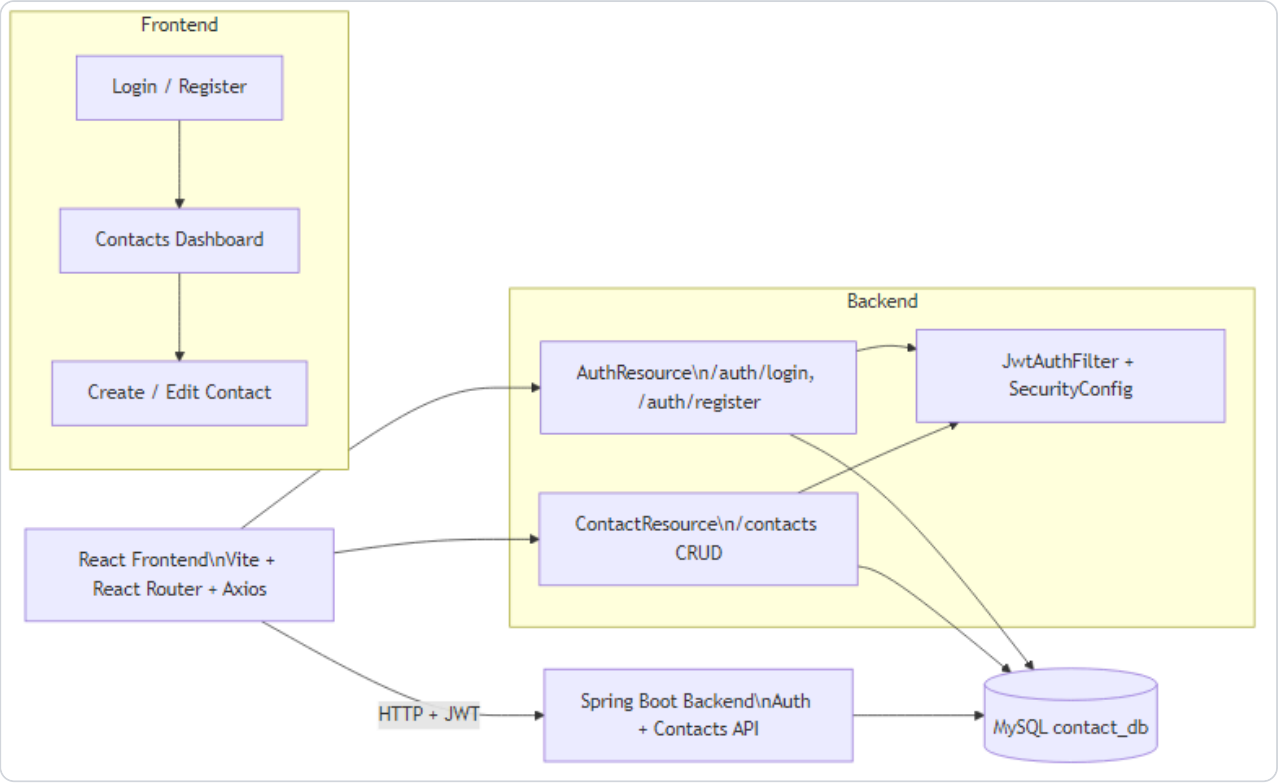
1. System Overview
2. Architecture Diagram
3. Authentication Flow Diagram
4. UI Screenshots
5. Backend Documentation
6. Frontend Documentation
7. Database Design
8. API Reference
9. Data Initialization (Demo User)
10. Run Instructions
11. Troubleshooting

1) System Overview

The application provides secure CRUD operations for contacts. Authentication is handled through JWT tokens. Every contact is linked to a specific user, ensuring users only access their own data.

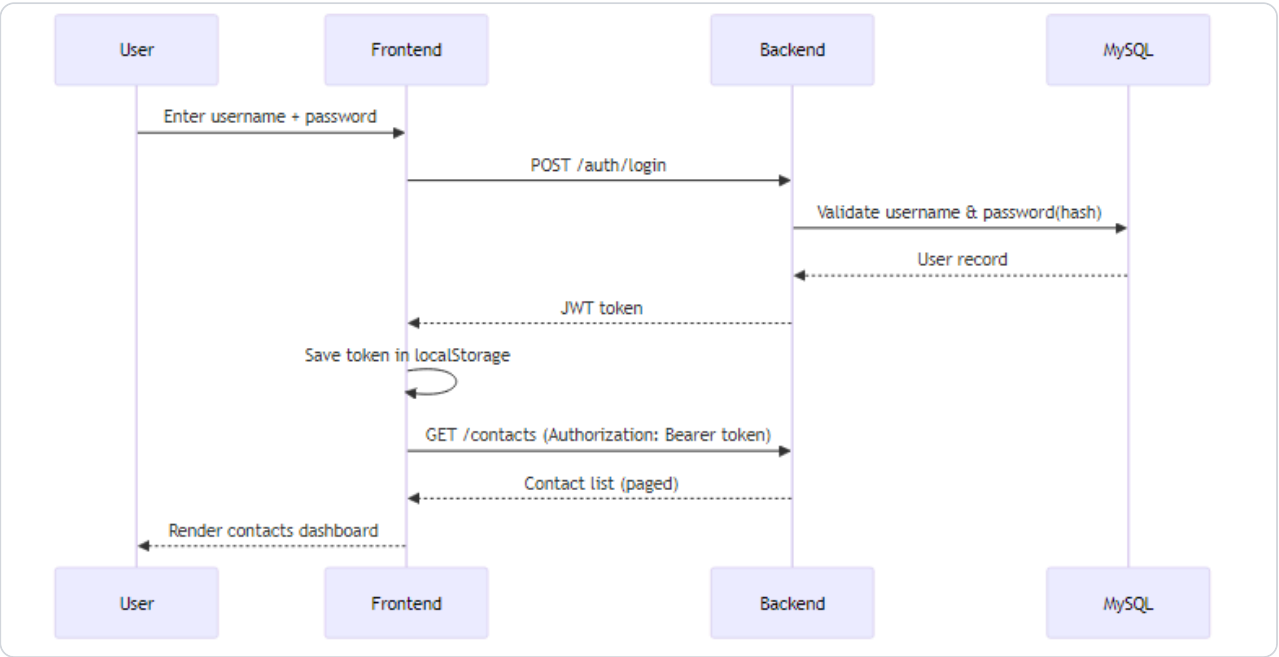
- **Authentication:** Register and login using username/password.
- **Authorization:** Protected endpoints require Bearer token.
- **Contacts Module:** Create, read, update, delete contacts.
- **Pagination:** Contact listing uses pageable backend response.

2) Architecture Diagram



Shows interaction among React frontend, Spring Boot backend, and MySQL database.

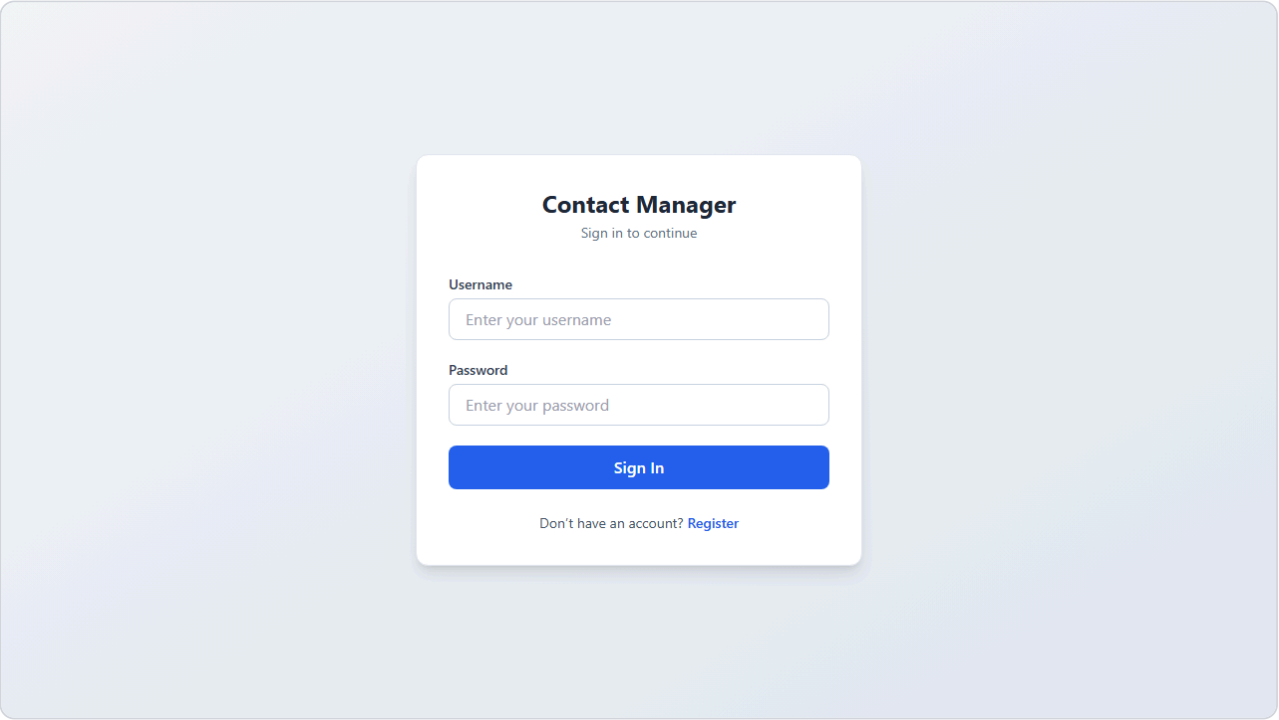
3) Authentication Flow Diagram



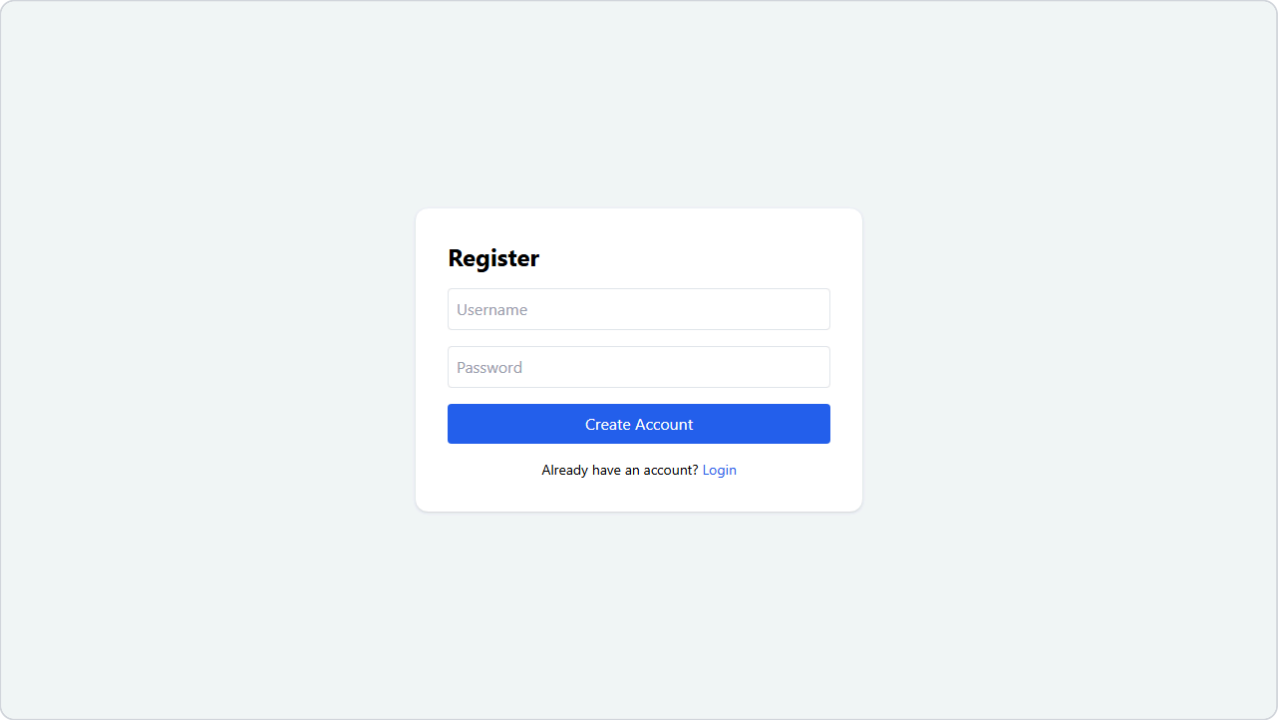
Sequence of login, JWT creation, token storage, and authenticated API access.

4) UI Screenshots

Login Page



Register Page



Contacts Dashboard

My Contacts

+ Add Contact Logout

Carla Mendes
carla.mendes@example.com
+1 555-0103
Edit Delete

Brian Smith
brian.smith@example.com
+1 555-0102
Edit Delete

Alice Johnson
alice.johnson@example.com
+1 555-0101
Edit Delete

Kondru Thirumalesh
kondruthirumalesh@gmail.com
9133462056
Edit Delete

5) Backend Documentation

Technology Stack

- Java 17
- Spring Boot 3.1.5
- Spring Data JPA + Hibernate
- Spring Security + JWT (jjwt 0.11.5)
- MySQL Connector

Key Backend Packages

Package	Purpose
`resource`	REST endpoints (`AuthResource` , `ContactResource`)
`security`	JWT generation/validation and security filter chain
`service`	Business logic for contacts
`repo`	JPA repositories for `User` and `Contact`
`domain`	Entity classes mapped to database tables
`config`	CORS, password encoder, data initializer

Security Notes

- `/auth/**` endpoints are public.
- All other endpoints are protected and require JWT.
- Frontend includes token as `Authorization: Bearer <token>` .

6) Frontend Documentation

Technology Stack

- React 19 + Vite
- React Router
- Axios API client with request interceptor for JWT
- Tailwind CSS utilities for styling

Frontend Flow

- User logs in from `/login` .
- JWT is saved in browser storage.
- Protected routes validate token existence.
- Contacts page fetches `/contacts?page=0&size=10` .

7) Database Design

users

Column	Type	Description
id	BIGINT	Primary key
username	VARCHAR (unique)	Login identifier
password	VARCHAR	BCrypt hash (not plaintext)
role	VARCHAR	Usually `USER`

contacts

Column	Type	Description
id	UUID	Primary key
name	VARCHAR	Contact name
email	VARCHAR	Email address
phone	VARCHAR	Phone number
title	VARCHAR	Professional title
address	VARCHAR	Address
status	VARCHAR	ACTIVE / INACTIVE
photo_url	VARCHAR	Optional avatar URL
user_id	BIGINT (FK)	Owner user id

8) API Reference

Method	Endpoint	Auth	Description
POST	`/auth/register`	No	Create account
POST	`/auth/login`	No	Get JWT token
GET	`/contacts`	Yes	Get paged contacts
POST	`/contacts`	Yes	Create contact
GET	`/contacts/{id}`	Yes	Get one contact
PUT	`/contacts/{id}`	Yes	Update contact
DELETE	`/contacts/{id}`	Yes	Delete contact

9) Data Initialization (Demo User)

On backend startup, the data initializer ensures a demo account exists and seeds sample contacts if missing.

- **Demo username:** `test`

- **Demo password:** `test123`
- Seeded contacts: Alice Johnson, Brian Smith, Carla Mendes
- Seeding is idempotent (safe across restarts)

10) Run Instructions

Backend

Open terminal in `contact-backend` and run:

`mvn spring-boot:run`

Backend URL: `http://localhost:8080`

Frontend

Open terminal in `contact-frontend` and run:

`npm install`
`npm run dev`

Frontend URL: `http://localhost:5173`

Database Credentials

Edit backend config file:

`contact-backend/src/main/resources/application.yml`

Set your own MySQL username/password under `spring.datasource`.

11) Troubleshooting

- If login fails after restart, clear old token from browser storage and login again.
- If backend fails to start, verify MySQL service and database credentials.
- If frontend is blank, check browser console and Vite terminal output.
- If ports are busy, stop existing processes on `8080` and `5173`.

Prepared for academic/demo use. This document can be shared with students for setup, usage, and architecture understanding.